

TP - Sécuriser l'accès à une base de données

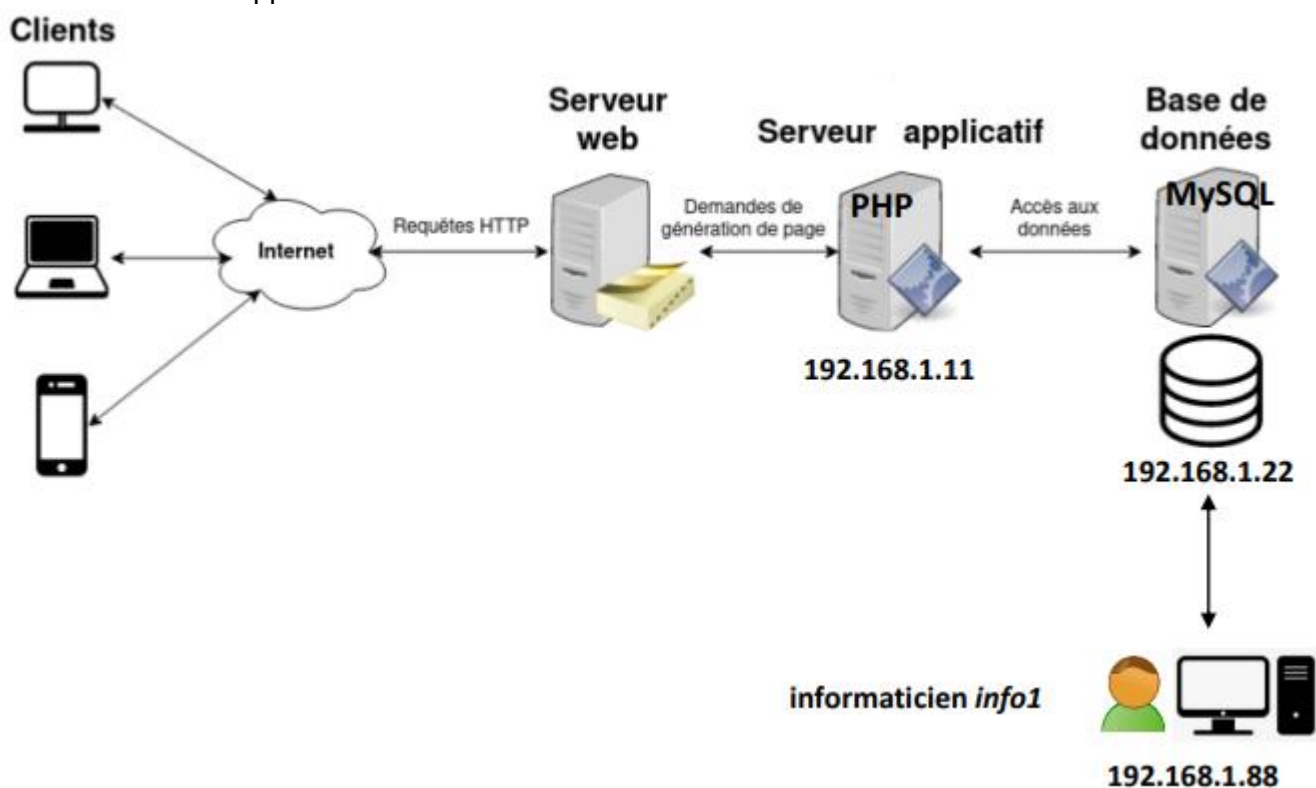
SIO 1 – BLOC 3 – Cybersécurité – Thème 3 – Sécuriser les équipements et les usages des utilisateurs

1 L'ajout d'utilisateurs à une base de données

L'entreprise GSB est un laboratoire pharmaceutique qui fabrique et commercialise des médicaments dans le monde entier.

Un site web, destiné aux professionnels de santé, permet de consulter les médicaments proposés. L'application web est écrite en PHP et la base de données qui enregistre les informations est sur le SGBDR (Système de gestion de base de données relationnelle) MySQL.

L'architecture de l'application est la suivante :



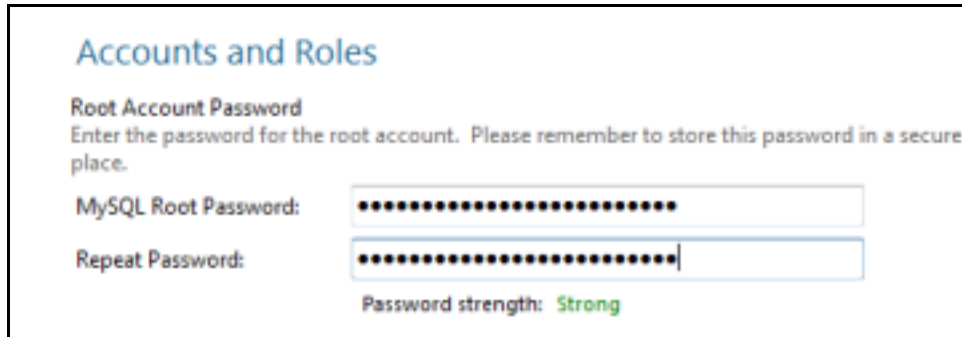
Qu.1.1 – Comment peut-on sécuriser l'accès à la base de données (c'est-à-dire empêcher que n'importe qui fasse n'importe quoi avec les données) :

Limiter l'accès à la base avec un identifiant et un mot de passe.

Donner des privilèges (=droits d'accès)

Filtrer sur les adresses IP

Lorsque l'administrateur de la base de données a installé MySQL, il a renseigné les informations suivantes :



Accounts and Roles

Root Account Password
Enter the password for the root account. Please remember to store this password in a secure place.

MySQL Root Password:

Repeat Password:

Password strength: **Strong**

Qu.1.2 – Qu'est-ce que le « MySQL root password » ?

C'est le mot de passe pour l'administrateur root de MySQL, il a TOUT les droits

Qu.1.3 – En TP avec easyPHP ou WAMPServer:

- quel compte utilisez-vous pour MySQL ?

root

- quel mot de passe utilisez-vous ?

(aucun)

- Commentez ce choix en termes de sécurité :

Sécurité nulle, il faut changer l'utilisateur afin d'avoir autre chose que root et mettre un mot de passe

Plusieurs personnes ou applications ont besoin d'accéder à MySQL.

L'administrateur va ajouter ces utilisateurs à MySQL et leur donner accès aux données requises, au moyen de commandes SQL.

L'instruction de MySQL pour créer un utilisateur est :

```
CREATE USER 'nom_user'@'adresseIP_user' IDENTIFIED BY 'password';
```

Qu.1.4 – Ecrire les requêtes SQL suivantes :

- Req 1 : crée l'utilisateur 'info1' (avec son adresse IP 192.168.1.88) avec le mot de passe 'Ut1IS3cr3t_88';

```
CREATE USER 'info1'@'192.168.1.88' IDENTIFIED BY 'Ut1IS3cr3t_88';
```

- Req 2 : crée l'utilisateur 'php', qui représente le programme PHP du serveur applicatif, avec le mot de passe 'MDPAppli#3547'.

```
CREATE USER 'php'@'192.168.1.11' IDENTIFIED BY 'MDPAppli#3547';
```

2 Gérer les privilèges des utilisateurs

2.1 Gestion des privilèges des utilisateurs

L'administrateur de la base va maintenant gérer les privilèges des utilisateurs, c'est-à-dire les droits d'accès aux différentes bases.

Le SGBDR MySQL comporte les bases de données suivantes :



Les instructions SQL pour donner des privilèges sont les suivantes :

Pour donner tous les privilèges, sur toutes les bases et toutes leurs tables à l'utilisateur nom_user, on utilise :

```
GRANT ALL PRIVILEGES ON * . * TO 'nom_user'@'adressesIP_user';
```

Pour donner une permission (= un privilège) en particulier, on utilise :

```
GRANT type_of_permission ON database_name.table_name TO 'username'@'adressesIP';
```

(type_of_permission peut être entre autres : SELECT, UPDATE, DELETE...)

Travail à faire :

Qu.2.1 – Ecrire la requête (Req 3) qui donne tous les privilèges (pour toutes les bases) à l'utilisateur 'info1' :

```
GRANT ALL PRIVILEGES ON * . * TO 'info1'@'192.168.1.88';
```

Qu.2.2 – L'application web ne doit pouvoir accéder qu'à la base 'pharma', et pas aux autres. Ecrire la requête (Req 4) qui donne à l'utilisateur 'php'tous les privilèges pour toutes les tables de la base 'pharma'.

```
GRANT ALL PRIVILEGES ON pharma.* TO 'php'@'192.168.1.11';
```

En fait, après réflexion, l'utilisateur 'info1' ne doit pas avoir accès à la base de données 'RH' qui contient des données personnelles. La commande pour révoquer une permission est :

```
REVOKE type_of_permission ON database_name.table_name FROM 'username'@'adressesIP';
```

Qu.2.3 – Travail à faire : Révoquer toutes les permissions de l'utilisateur 'info1' sur la base 'RH'.

```
REVOKE ALL PRIVILEGES ON RH.* FROM 'info1'@'192.168.1.88';
```

2.2 Exercice

Un nouvel utilisateur 'employe_RH' vient d'arriver au service Ressources humaines. Il utilise le poste d'adresse IP 192.168.1.99. Il doit pouvoir accéder à la base 'RH' seulement, et pour des opérations de consultation uniquement (SELECT). Son mot de passe est 'kim-3-RH@2023'.

Qu.2.4 – Travail à faire : écrire les requêtes SQL qui permettent d'ajouter l'utilisateur 'employe_RH' et de lui octroyer la permission de consultation sur la base 'RH' uniquement.

```
CREATE USER 'employe_RH'@'192.168.1.99' IDENTIFIED BY 'kim-3-RH@2023';  
GRANT SELECT ON RH.* TO 'employe_RH'@'192.168.1.99';
```

Qu.2.5 – Travail à faire : quel est l'intérêt pour la sécurité de préciser l'adresse IP de l'utilisateur employe_RH, c'est-à-dire 192.168.1.99, lorsque l'on crée cet utilisateur ?

Pour indiquer à la DB que l'employé se connecte uniquement avec cette adresse IP, si une autre adresse IP utilise cet utilisateur, la DB ne l'acceptera pas

Cet utilisateur quitte finalement le service RH et n'a plus besoin d'accéder à la base de données. Il faut le supprimer de MySQL. Pour supprimer un utilisateur, la commande SQL est :

```
DROP USER 'username'@'adresseIP';
```

Qu.2.6 – Travail à faire : écrire la requête SQL qui supprime l'utilisateur 'employe_RH'

```
DROP USER 'employe_RH'@'192.168.1.99';
```

Qu.2.7 – Travail à faire : écrire les requêtes SQL qui permettent d'ajouter l'utilisateur 'employe_gestion' avec le poste 192.168.1.100 et de lui octroyer les droits de lecture, ajout et modification sur la base gestion.

```
CREATE USER 'employe_gestion'@'192.168.1.100' IDENTIFIED BY 'kim-3-RH@2023';  
GRANT SELECT, UPDATE, DELETE ON gestion.* TO 'employe_RH'@'192.168.1.99';
```